

Nahamcon 2020 Crypto - Raspberry

deuterium — 2020-06-14

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/nahamconCTF/crypto/Raspberry/2020-06-14-Nahamcon-2020-Raspberry.html>

Abstract

CTF writeup: NahamCon 2020's Raspberry packs RSA with snack-sized primes. Factor them, compute the totient and unwrap the ciphertext.

Raspberry

Challenge

169 Solves



Raspberry

125

Raspberries are so tasty. I have to have more than just one!

Download the file below.

 [prompt.txt](#)

Flag

Submit

We are provided with [prompt.txt](#), the contents of which read

```
n =
77352089398489850796806146335817822743711481572933529049053133154094184673227267028481
89532721490121708517697848255948254656192793679424796954743649810878292688507385952920
22948377638992265038873997507258766086698660308098698035921952511158965919117293704786
9008331982383695605801970189336227832715706317
e = 65537
```

```

C =
53007317095837144510629052385319721605185250808580951845818393666800229952978630139116
12079520115435945472004626222058696229239285358638047675780769773922795279074074633888
72078719554954483529152811609390945622567015273319155665063955390619585697979427334959
8903501654956482056938935258794217285615471681

```

Lets again head to [alpetron](#), suspecting there may be a lot of small prime factors

Electronics Mathematics Programs Contact
ESP

Integer factorization calculator

[Alpetron](#) > [Programs](#) > Integer factorization calculator

Value

96954743649810878292688507385952920229483776389922650388739975072587660866986603080986980359219525111589659191172937047869008331982383695605801970189362278327157063

One numerical expression or loop per line. Example: x=3;x=n(x);c<=100;x-1

Only evaluate
Factor
Help
Config
Wizard

Press the **Help** button to get help about this application. Press it again to return to the factorization. Keyboard users can press CTRL+ENTER to start factorization. This is the WebAssembly version.

- 7735 208939 848985 079680 614633 581782 274371 148157 293352 904905 313315 409418 467322 726702 848189 532721 490121 708517 697848 255948 254656 192793 679424 796954 743649 810878 292688 507385 952920 229483 776389 922650 388739 975072 587660 866986 603080 986980 359219 525111 589659 191172 937047 869008 331982 383695 605801 970189 336227 832715 706317 (304 digits) = 2208 664111 × 2214 452749 × 2259 012491 × 2265 830453 × 2372 942981 × 2393 757139 × 2465 499073 × 2508 863309 × 2543 358889 × 2589 229021 × 2642 723827 × 2758 626487 × 2850 808189 × 2947 867051 × 2982 067987 × 3130 932919 × 3290 718047 × 3510 442297 × 3600 488797 × 3644 712913 × 3650 456981 × 3726 115171 × 3750 978137 × 3789 130951 × 3810 149963 × 3979 951739 × 4033 877203 × 4128 271747 × 4162 800959 × 4205 130337 × 4221 911101 × 4268 160257

Number of divisors: 4294 967296

Sum of divisors: 7735 209020 785957 803608 477795 574261 494558 463953 002507 901977 035607 471184 250817 511717 583963 020477 550818 601353 018134 226671 494401 602457 563130 715324 072885 286128 235636 998769 006853 037774 540316 259560 644962 442296 696989 757297 912511 603160 122033 236254 622846 793241 143602 786655 530543 380539 795308 299632 466984 960000 000000 (304 digits)

Euler's totient: 7735 208858 912013 174807 786153 875265 311232 475601 814043 799113 971819 203978 928431 722767 684674 067119 725000 291122 190443 795615 639597 996271 053342 184657 566093 781734 940605 689774 479814 988576 374248 690133 868658 052837 262849 840790 750991 863632 288857 652325 134901 612884 172359 123108 137383 953436 272360 300748 996608 000000 000000 (304 digits)

Which almost instantly factors the N using [Elliptic Curve Method](#) indicating my suspicion was true.

Now we have a lot of prime factors, one may simply grab the Euler's Totient, I however prefer working over primes to demonstrate how its calculated

```

from gmpy2 import invert
from math import prod
n =
77352089398489850796806146335817822743711481572933529049053133154094184673227267028481
89532721490121708517697848255948254656192793679424796954743649810878292688507385952920
22948377638992265038873997507258766086698660308098698035921952511158965919117293704786
9008331982383695605801970189336227832715706317
e = 65537
c =
53007317095837144510629052385319721605185250808580951845818393666800229952978630139116
12079520115435945472004626222058696229239285358638047675780769773922795279074074633888
72078719554954483529152811609390945622567015273319155665063955390619585697979427334959
8903501654956482056938935258794217285615471681

factors =
[2208664111, 2214452749, 2259012491, 2265830453, 2372942981, 2393757139, 2465499073, 25088633
09, 2543358889, 2589229021, 2642723827, 2758626487, 2850808189, 2947867051, 2982067987, 313093
2919, 3290718047, 3510442297, 3600488797, 3644712913, 3650456981, 3726115171, 3750978137, 3789
130951, 3810149963, 3979951739, 4033877203, 4128271747, 4162800959, 4205130337, 4221911101, 42
68160257]

phi = prod(i-1 for i in factors) # cool new function in python3.8 ;)
# just take the product of prime - 1 for all primes to calculate phi
# since no prime is repeated

```

```
d = invert(e,phi)
m = pow(c,d,n)
print(bytes.fromhex(hex(m)[2:]).decode())
```

If you got stuck over the challenge, I would suggest reading [wikipedia](#) for more information.

~~flag-there are a few extra berries in this one~~

| Interactive controls are omitted from this PDF. See the canonical HTML version.